

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250279966

Kind Code

A1

Publication Date

September 04, 2025

Inventor(s)

LIAO; Qi et al.

MICROSERVICE-AWARE FRAMEWORK FOR NETWORK-APPLICATION COLLABORATION

Abstract

Method comprising receiving, for each of plural microservices, a respective requirement of a quality of service, wherein the plural microservices jointly build an application; receiving a first label, wherein the first label indicates a policy status of a communication system conveying data packets from the microservices for the application; receiving an indication of a first quality of experience of the application; determining, for a first one of the microservices, an importance level based on the requirement of the quality of service of the first one of the microservices, the first label, and the first quality of experience, marking a data packet originated from or designated to the first one of the microservices and to be conveyed by the communication system by the importance level determined for the first one of the microservices; providing the data packet marked by the importance level to the communication system for conveying the data packet.

Inventors: LIAO; Qi (Stuttgart, DE), LIETO; Alessandro (Stuttgart, DE), PAPILLON; Serge (Massy, FR)

Applicant: Nokia Solutions and Networks Oy (Espoo, FI)

Family ID: 94478495

Appl. No.: 19/065597

Filed: February 27, 2025

Foreign Application Priority Data

FI	20245259	Mar. 01, 2024
----	----------	---------------

Publication Classification

Int. Cl.: H04L47/2475 (20220101); H04L41/5067 (20220101)

U.S. Cl.:

Background/Summary

TECHNICAL FIELD

[0001] The present disclosure relates to collaboration of a network with an application composed of microservices.

ABBREVIATIONS

[0002] 3GPP 3.sup.rd Generation Partnership Project [0003] 5G 5.sup.th Generation [0004] ADAS Advanced Driver Assistance Systems [0005] AI Artificial Intelligence [0006] API Application Programming Interface [0007] AR Augmented Reality [0008] CSC Communications Service Customer [0009] CSP Communications Service Provider [0010] DDPG Deep Deterministic Policy Gradient [0011] ECN Explicit Congestion Notification [0012] IMO Importance Marker Optimizer [0013] KPI Key Performance Indicators [0014] LAN Local Area Network [0015] MAC Medium Access Control [0016] MF Max-Min Fairness [0017] ML Machine Learning [0018] NSaaS Network Slice as a Service [0019] NSMF Network Slice Management Function [0020] NSSMF Network Slice Subnet Management Function [0021] PF Proportional Fair [0022] PHY Physical Layer [0023] QoE Quality of Experience [0024] Qos Quality of Service [0025] RAN Radio Access Network [0026] RR Round Robin [0027] SLA Service Level Agreement [0028] TD Time Domain [0029] URLLC Ultra Reliable Low Latency Communications [0030] VR Virtual Reality

BACKGROUND

[0031] In 3GPP, a service level agreement (SLA) is a contract between the communications service provider (CSP) and communications service customer (CSC) defining the communication service requirements for defined services, such as 5G LAN, ultra reliable low latency communications (URLLC) [3GPP TS 28.535]. This means that the application services can specify the minimum acceptable standard of network services and the CSP provides the required network services for them. Examples of application services are the streaming service Netflix, or the cloud gaming service GeForce NOW, or automated driving service TomTom's driver assistance systems (ADAS). [0032] Today's applications are usually composed of a multitude of microservices managed through virtual machines or container environments. Enterprises such as Amazon, Netflix, and Uber have already adopted microservices architecture to achieve scalability and adaptability advantages. [0033] There are several solutions for the management of a microservice architecture for cloud-native applications. The most successful platform adopted for the design and deployment of cloud applications is Kubernetes, which offers a wide range of solutions for the optimal placement, selection, and execution of the microservices which compose the applications.

SUMMARY

[0034] It is an object to improve the prior art.

[0035] According to a first aspect, there is provided an apparatus comprising [0036] means for receiving, for each of plural microservices, a respective requirement of a quality of service, wherein the plural microservices jointly build an application; [0037] means for receiving a first label, wherein the first label indicates a policy status of a communication system conveying data packets from the microservices for the application; [0038] means for receiving an indication of a first quality of experience of the application; [0039] means for determining, for a first one of the microservices, an importance level based on the requirement of the quality of service of the first one of the microservices, the first label, and the first quality of experience, [0040] means for marking a data packet originated from or designated to the first one of the microservices and to be conveyed by the communication system by the importance level determined for the first one of the microservices; [0041] means for providing the data packet marked by the importance level to the

communication system for conveying the data packet.

[0042] The means for determining may be configured to determine, for each of the microservices, a respective importance level based on the requirement of the quality of service of the respective microservice, the first label, and the first quality of experience, and [0043] the means for marking may be configured to mark, for each of the microservices, a data packet originated from or designated to the respective microservice and to be conveyed by the communication system by the importance level determined for the respective microservice.

[0044] The means for determining may be configured to determine an inter-microservice importance level based on the requirement of the quality of service of the first one of the microservices, the requirement of the quality of service of a second one of the microservices, the first label, and the first quality of experience; and [0045] the means for marking may be configured to mark an inter-microservices data packet by the inter-microservice importance level, wherein the inter-microservices data packet is to be conveyed by the communication system between the first one of the microservices and the second one of the microservices.

[0046] The importance level determined for the first one of the microservices may be included in a header of the data packet originated from or designated to the first one of the microservices and to be conveyed by the communication system.

[0047] The data packet originated from or designated to the first one of the microservices and to be conveyed by the communication system may belong to a traffic flow, and the traffic flow may be marked by the importance level.

[0048] The apparatus may further comprise [0049] means for receiving, from the communication system in response to the providing the data packet marked by the importance level to the communication system, a second label indicating a policy status of the communication system after the communication system evaluated the importance level; [0050] means for receiving, in response to the providing the data packet marked by the importance level to the communication system, an indication of a second quality of experience of the application; [0051] means for adapting the determining by the means for determining based on the first label, the second label, the requirements of the quality of service of each of the microservices, the first quality of experience, and the second quality of experience for optimizing the quality of experience of the application.

[0052] The means for adapting may be configured to adapt the determining by the means for determining by machine learning. The machine learning may comprise at least one of reinforcement learning and supervised learning.

[0053] According to a second aspect, there is provided an apparatus, comprising [0054] means for receiving, from an optimizer, a data packet for conveying the data packet by a communication system, wherein the data packet is marked by an importance level; [0055] means for modifying a policy status of the communication system based on the importance level; [0056] means for informing the optimizer by a label on the policy status of the communication system after the policy status of the communication system was modified by the means for modifying.

[0057] The apparatus may further comprise [0058] means for mapping the policy status of the communication system to the label, wherein a number of available labels is smaller than a number of available policy statuses of the communication system.

[0059] The policy status of the communication system may comprise at least one of a policy status of a physical layer of the communication system, a policy status of a medium access control layer of the communication system, a policy status of a transport layer of the communication system, an edge computing policy status of the communication system, or a computational resource allocation policy status of the communication system.

[0060] According to a third aspect, there is provided a method comprising [0061] receiving, for each of plural microservices, a respective requirement of a quality of service, wherein the plural microservices jointly build an application; [0062] receiving a first label, wherein the first label indicates a policy status of a communication system conveying data packets from the microservices

for the application; [0063] receiving an indication of a first quality of experience of the application; [0064] determining, for a first one of the microservices, an importance level based on the requirement of the quality of service of the first one of the microservices, the first label, and the first quality of experience, [0065] marking a data packet originated from or designated to the first one of the microservices and to be conveyed by the communication system by the importance level determined for the first one of the microservices; [0066] providing the data packet marked by the importance level to the communication system for conveying the data packet.

[0067] For each of the microservices, a respective importance level may be determined based on the requirement of the quality of service of the respective microservice, the first label, and the first quality of experience, and, [0068] for each of the microservices, a data packet originated from or designated to the respective microservice and to be conveyed by the communication system may be marked by the importance level determined for the respective microservice.

[0069] An inter-microservice importance level may be determined based on the requirement of the quality of service of the first one of the microservices, the requirement of the quality of service of a second one of the microservices, the first label, and the first quality of experience; and [0070] an inter-microservices data packet may be marked by the inter-microservice importance level, wherein the inter-microservices data packet is to be conveyed by the communication system between the first one of the microservices and the second one of the microservices.

[0071] The importance level determined for the first one of the microservices may be included in a header of the data packet originated from or designated to the first one of the microservices and to be conveyed by the communication system.

[0072] The data packet originated from or designated to the first one of the microservices and to be conveyed by the communication system may belong to a traffic flow, and the traffic flow may be marked by the importance level.

[0073] The method may further comprise [0074] receiving, from the communication system in response to the providing the data packet marked by the importance level to the communication system, a second label indicating a policy status of the communication system after the communication system evaluated the importance level; [0075] receiving, in response to the providing the data packet marked by the importance level to the communication system, an indication of a second quality of experience of the application; [0076] adapting the determining the importance level based on the first label, the second label, the requirements of the quality of service of each of the microservices, the first quality of experience, and the second quality of experience for optimizing the quality of experience of the application.

[0077] The determining the importance level may be adapted by machine learning. The machine learning may comprise at least one of reinforcement learning and supervised learning.

[0078] According to a fourth aspect, there is provided a method, comprising [0079] receiving, from an optimizer, a data packet for conveying the data packet by a communication system, wherein the data packet is marked by an importance level; [0080] modifying a policy status of the communication system based on the importance level; [0081] informing the optimizer by a label on the policy status of the communication system after the policy status of the communication system was modified.

[0082] The method may further comprise [0083] mapping the policy status of the communication system to the label, wherein a number of available labels is smaller than a number of available policy statuses of the communication system.

[0084] The policy status of the communication system may comprise at least one of a policy status of a physical layer of the communication system, a policy status of a medium access control layer of the communication system, a policy status of a transport layer of the communication system, an edge computing policy status of the communication system, or a computational resource allocation policy status of the communication system.

[0085] Each of the methods of the third and fourth aspects may be a method of collaboration of a

network with an application.

[0086] According to a fifth aspect, there is provided a computer program product comprising a set of instructions which, when executed on an apparatus, is configured to cause the apparatus to carry out the method according to any of the third and fourth aspects. The computer program product may be embodied as a computer-readable medium or directly loadable into a computer.

[0087] According to some example embodiments, at least one of the following advantages may be achieved: [0088] Network may provide resources and capacity according to the importance of the microservices for the QoE; [0089] Resources and energy may be utilized efficiently; [0090] Overprovisioning may be avoided or reduced; [0091] Privacy of at least one of the application and the network (or both or them) towards the other one may be preserved.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0092] Further details, features, objects, and advantages are apparent from the following detailed description of the preferred example embodiments which is to be taken in conjunction with the appended drawings, wherein:

[0093] FIG. 1 shows a framework according to some example embodiments;

[0094] FIG. 2 (comprising FIGS. 2A and 2B) shows an action flow according to some example embodiments;

[0095] FIG. 3 shows importance marker assignment extended to inter-microservices communication according to some example embodiments;

[0096] FIG. 4 shows a basic actor-critic architecture for optimizing the importance level assignment according to some example embodiments;

[0097] FIG. 5 shows how an example embodiment is applied to a streaming application (e.g. Netflix) as an example;

[0098] FIG. 6 shows an apparatus according to an example embodiment;

[0099] FIG. 7 shows a method according to an example embodiment;

[0100] FIG. 8 shows an apparatus according to an example embodiment;

[0101] FIG. 9 shows a method according to an example embodiment; and

[0102] FIG. 10 shows an apparatus according to an example embodiment.

DETAILED DESCRIPTION OF CERTAIN EXAMPLE EMBODIMENTS

[0103] Herein below, certain example embodiments are described in detail with reference to the accompanying drawings, wherein the features of the example embodiments can be freely combined with each other unless otherwise described. However, it is to be expressly understood that the description of certain example embodiments is given by way of example only, and that it is by no way intended to be understood as limiting the disclosure to the disclosed details.

[0104] Moreover, it is to be understood that the apparatus is configured to perform the corresponding method, although in some cases only the apparatus or only the method are described.

[0105] The SLA is basically a one-directional framework, i.e., CSC first specifies requirements, then CSP provides network service. This approach may lead to the following challenges: [0106] 1) Today's widely used applications are usually built as collections of multiple microservices which can operate independently and communicate via well-defined interfaces typically using lightweight APIs. Each of these microservices has its own network service requirements, and contributes to the final user experience (QoE) with different levels of importance. A network slicing-based SLA cannot address the different requirements and importance levels of multiple microservices within an application. [0107] 2) The one-directional framework assumes that the customer/application knows exactly how to specify the requirements to provide optimal user experience. However, in real life, it remains unknown to both application and network how the network QoS maps to the

users' QoE, due to the limited interactions between the CSP and CSC. This usually leads to an over-provision of communications resources, which jeopardizes the network's performance in a multi-user, resource-hungry scenario, such as the cloud-gaming and AR/VR.

[0108] The decomposition of the application into microservices introduces some challenges to the network service: [0109] 1) The operation of different microservices may result in different requirements on the communication system (network). E.g., for Uber, the microservices for trip management and billing have a higher latency and higher security requirements than the passenger and driver management (ratings and reviews). [0110] 2) The communication between the microservices may result in different network requirements. Some microservices require synchronous communication while others allow asynchronous communication or event driven communication. E.g., for Netflix the user authentication requires synchronous communication between the authentication microservice and the user management microservice, while the ratings and recommendations allow asynchronous communication. [0111] 3) For a good user's experience, a correct and timely execution of each microservice is typically needed. However, not all the microservices contribute homogeneously to the user's experience. E.g., the delay of the rating/reviews does not impact the users' experience as strongly as the delay for ordering or for payment. A differentiation among different microservices in level of importance can help the network to offer dedicated and prioritized performance for the ones that are the most critical for the user experience.

[0112] Thus, it is useful to introduce a micro service-aware communications service solution targeting goal-oriented user experience. As additional requirements, it would be useful that the interaction between the network and the application is lightweight. Still furthermore, it is recommendable to preserve privacy of both sides (application (CSC) and communication service (CSP)).

[0113] Solutions for the management of microservices-based applications (such as Kubernetes) typically consider only the cloud infrastructure, performing actions based on the availability and performance of computing and storage nodes. They offer customizable ways to handle the microservices, by defining both QoS and priority levels that are used by a scheduler to optimally allocate resources in the cloud infrastructure. Those solutions typically lack knowledge from the network side. The lack of knowledge might result in overprovisioning on the network side, which can be very costly for ensuring low-latency and ultra reliability, like for industrial applications. Additional services offer network monitoring as a service, but they cannot take actions at network level, while rather responding to any network impairment.

[0114] Some example embodiments take into account both the inherent microservice architecture behind an application and how the different microservices are valued/prioritized differently based on the expected goal of the application (i.e. QoE of the users of the application). To ensure a good (optimized) QoE, at least some of the packets conveyed for the application (i.e. from/to the microservices building the application and between the microservices) by the communication system are marked by an importance level. The communication system (sometimes denoted as network) takes the importance level into account for treating the packet and to adapt its status (network policy) accordingly.

[0115] A conventional "one SLA for one application/service" approach does not take into account that the microservices may have different QoS requirements. This ignorance may limit the scalability and profitability of the network service. Moreover, the one-directional framework of a conventional SLA (i.e., "the application specifies requirements, and network provides service") often causes over-provision of the communications resources and inefficient use of the energy. Especially in a resource-hungry multi-user scenario such as cloud gaming or AR/VR, network can easily suffer from congestion because of over-provision for a lot of applications.

[0116] Some example embodiments provide a lightweight, microservice-aware, dual-side privacy-preserving network-application collaborative framework to improve the goal-oriented user's QoE.

In particular, the following problems are addressed: [0117] Achieving the user's goal: How to autonomously learn the importance levels that reflect different QoS requirements of multiple microservices and their different contributions to the final QoE? [0118] Application-Network information exchange: How to facilitate the microservice-specific information exchange between the application provider and the network provider, while preferably protecting the privacy of both sides? [0119] Lightweight Application-Network cooperative framework: [0120] From the application side, how to assign the importance levels to microservices and efficiently make them aware to the network? [0121] From the network side, how to enable the importance level-aware network optimization policies and efficiently make them aware to the application?

[0122] A framework according to some example embodiments is shown in FIG. 1. In FIG. 1, the numerals in circles show where the actions depicted in FIG. 2 (described further below) take place. The components of this framework are as follows: [0123] On application/service side: [0124] Each application/service assigns different importance levels to its microservices and embeds this "importance marker" in the header of the network packets and/or traffic flows. As another option, the application may signal the importance marker to the network orchestration and management function to identify the flow based on their importance marker. Thus, all the packets of the flow are marked by the importance marker of the flow. Note that the importance marker can change over time to adapt to the current goal of the application. [0125] An Importance Marker Optimizer (IMO) assigns the importance level for each microservice using techniques such as reinforcement learning based on the following information: [0126] Goal-oriented reward: observed users' QoE [0127] Microservice-specific state: microservice-specific QoS requirements [0128] Extracted network policy status: a "network label" (extracted label of network's policies, provided by the network). [0129] The network need not to know any details of the microservices and is not involved in the application's decision on the assignment of the importance levels to the microservices. This preserves the privacy of the application towards the network. [0130] On communications system side: [0131] Network owns a set of optional policies to allocate resources based on the "importance marker", e.g., channel allocation, retransmission probability control, radio resource scheduling, and other resource allocation policies including both the communication and computational resources (e.g., in case of mobile edge computing). The differentiation among different microservices in level of importance helps the network to offer dedicated and prioritized performance for the ones that are most critical for the user experience. [0132] The network maps its selected policies (e.g., radio scheduling, congestion control, retransmission strategy, . . .) to a "network label", e.g., with categorical embedding, and provide it to the application. A network label could potentially correspond also to a specific network slice identifier, where a network slice embeds the E2E network policies adopted for serving the flows with the same importance marker.

[0133] In some example embodiments, the label may indicate the status of the communication system (network) in detail, such as a physical layer of the communication system, a status of a medium access control layer of the communication system, a status of a transport layer of the communication system, an edge computing policy status of the communication system, or a computational resource allocation policy status of the communication system. However, in some example embodiments, the label shows an aggregated status of the network. I.e., typically, the number of available labels is smaller than the number of status available for the network. This preserves the privacy of the network towards the application.

[0134] The importance marker and the label enable an ultra-light information exchange between the application and the network, since the network may only provide a high-level extracted label that represents its combination of policies (i.e., network configurations), to avoid a heavy and frequent feedback on its fine states such as channel states or buffer size, etc.

[0135] In some example embodiments, the framework is extended to inter-microservices communication (i.e. between different microservices of the application) conveyed by the communications system. The importance marker assigned to a packet in inter-microservices

communication may depend on the QoS requirements of both microservices.

[0136] In some example embodiments, the importance marker may be computed by IMO. IMO may be implemented as an add-on to the API gateway. As another option, IMO may be implemented as a stand-alone application or microservice. It helps the network to adapt its policies towards the application's goal (i.e. a good or even optimized QoE) by providing dedicated or prioritized performance to the microservices that are the most critical for the user's QoE.

[0137] FIG. 2 (comprising FIGS. 2A and 2B) shows an action flow according to some example embodiments. The numerals of the actions in FIG. 2 are also shown in FIG. 1 (in circles). In FIG. 1, the numerals indicate where the respective action takes place in the framework according to some example embodiments. The actions are as follows: [0138] 1. Network (e.g., the network management and orchestration function or a network slice management function (NSMF)) defines its own combination of the policies across different layers (e.g., channel allocation policy at the PHY layer, packet scheduling policy at the MAC layer, etc.) (1.a) and maps it to a network label, typically by an encoding technique (1.b), such as categorical embedding. The details of the “mapping from policy combination to network label” solution is explained further below. At least some of these policies, if not all of them, are aware of the importance markers. For example, according to a policy for channel allocation, a packet with higher importance marker will be allocated with a better channel; according to a policy for scheduling, a packet with a lower importance marker will be allocated with less resources and be considered to be more delay tolerant. An example design of an importance marker-aware network policy is provided further below. [0139] 2. Network (e.g., the network management and orchestration function or a network slice management function (NSMF)) sends its extracted network label (indicating its policy status) to the IMO on the application side. The IMO receives this message from the network. [0140] 3. Application collects the QoS requirements of the microservices building the application (3.a), e.g., via the API gateway. The application provides the information about the QoS requirements to the IMO for the importance marker assignment (3.b). [0141] 4. Application collects the users' feedback and compute the reward based on the goal-oriented user experience (QoE). An example how to determine a QoE is described for Netflix as an example application further below. The determination of the QoE is application specific. [0142] 5. Based on the collected data of network label, QoS requirements of the microservice, and observed user feedback (QoE), IMO computes the state (5.a), reward (5.b), and decides the actions of the importance level assignment to each microservice (5.c). E.g., in FIG. 1, the microservices 3, 2, and 1 are assigned with different importance markers 1, 3, and 2, respectively. Note that although in FIG. 1 ranked positive integers are used as the importance markers, other values can also serve as importance markers, such as positive floats. The optimized microservice-specific importance markers are then provided to the API gateway (5.d). [0143] IMO for importance marker assignment may be trained with AI/ML techniques, e.g., reinforcement learning. An example of the IMO based on reinforcement learning is provided further below. [0144] As another option, IMO may comprise a look-up table, where the importance marker is indicated for each combination of network label, QoS requirement of the microservice, and QoE. In some example embodiments, IMO may derive an intermediate value by a look-up table based on a subset of the input parameters network label, QoS requirement of the microservice, and QoE, and determine the importance marker by a trained neural network into which the intermediate value and the remaining input parameters are input. [0145] 6. The API gateway embeds the importance marker determined by IMO into the headers of the packets or traffic flows from the different microservices. These packets and traffic flows are transmitted and received over the communications system. [0146] 7. If the packets or traffic flows have importance markers, the network activates the importance marker-aware policies and applies them to the packets or traffic flows. In addition, the network adapts its policies (such as communications and/or computational resource allocation decisions) for the packets or traffic flows based on the importance markers. An example is described further below. [0147] 8. In some example

embodiments, the determination of the importance marker by IMO is static. However, according to some example embodiments, with the continuously collected samples (QoS requirements of the microservices, labels of the network, and QoEs) in IMO, the IMO can train/update/fine-tune its model according to the application's preference. Some options are as follows: [0148] a. The model is fine-tuned with the newly collected samples periodically, [0149] b. The model is continuously updated with continual learning, or [0150] c. The model retraining is activated reactively or predictively, by observing or predicting the degradation of users' QoE.

[0151] Not only the different microservices may have different requirements to the network, but also inter-microservice communication may have different network requirements. As an example, for Netflix the user authentication requires synchronous communication with more strict delay constraints between the authentication microservice and the user management microservice, while the ratings and recommendations allow asynchronous communication with more delay tolerant communication.

[0152] As shown in FIG. 3, the importance marker assignment may not only mark the packets from/to the microservices to/from an entity external to the application, but also the communication messages between the microservices of the application. For example, in FIG. 3, microservices 3, 2, and 1 are assigned with importance marker 1, 3, 2, respectively.

[0153] Moreover, the communication messages between microservices 2 and 3 are assigned with importance marker 2, while communication messages between microservice 1 and 2 are assigned with importance marker 1. Thus, the resource and energy efficiency for inter-microservice communications is improved, too.

[0154] For inter-microservice communication, except for the actions 3, 5, and 6, all the other steps remain the same as explained for FIGS. 1 and 2. The modifications of the actions 3, 5, and 6 are described below: [0155] Action 3: Application collects the QoS requirements for not only operating the microservices, but also the inter-microservice communication. [0156] Action 5: The IMO also includes the QoS requirements of the inter-microservice communications in the input features, and, in the output actions, also decides the importance level of the inter-microservice communications. [0157] Action 6: The optimized importance markers are assigned to both microservices' operating packets/flows and the messages for inter-microservice communications.

[0158] From the network side, the actions remain the same as described with respect to FIGS. 1 and 2 because the network policy adapts the priority or weight of the packet/flow based on the assigned importance markers, regardless of whether it is for operating the microservice or for inter-microservice communications.

[0159] Hereinafter, examples of network label extraction (action 1.b), a marker aware policy (action 7), and implementation of IMO are described at greater detail.

Network Label Extraction

[0160] Next, an example of network label extraction (action 1 in FIG. 2; i.e. how the network label is derived based on the actual policy status of the network) is described. In some example embodiments, the label will be created and managed at the Network Management and Orchestration Function level. As another option in the context of 5G slicing, the label management takes place in the NSMF (Network Slice Management Function), that relies on the different NSSMF (Network Slice Subnet Management Function) at Access Network, Transport Network and Core Network level to enforce the configuration and optimization policies of the network for the slices. The NSMF and NSSMF may be enhanced to create and maintain the network labels.

[0161] The network may be configured with a large set of network optimization policies composed of optional policies for each of the layers. For example, the options of the RAN scheduler at the MAC layer include round robin (RR), proportional fair (PF), max-min fairness (MF), etc. The network label mapping function may map the network's operating policies to a short code as the "network label", as shown in Block A in FIG. 1.

[0162] Assuming the network has a set of N policies implemented at different layers, denoted by a

set $\text{custom-character}=\{1, \dots, N\}$. Each policy $n \in \text{custom-character}$ has $K.\text{sub}.n$ different options $\text{custom-character}.\text{sup}.(n)=\{O.\text{sub}.1.\text{sup}.(n), \dots, O.\text{sub}.K.\text{sub}.n.\text{sup}.(n)\}$ to activate for operating the network. For example, the policy set custom-character includes the beamforming policy at PHY layer, the scheduling policy at MAC layer, etc., while the options of the scheduling policy include RR, PF, MF. A network operates under a set of chosen options of these policies $\text{custom-character}^*=\{\text{custom-character}.\text{sub}.*.\text{sup}.(1), \dots, \text{custom-character}.\text{sub}.*.\text{sup}.(N)\}$, where $\text{custom-character}.\text{sub}.*.\text{sup}.(n)$ denotes the chosen option for policy $n \in \text{custom-character}$. Thus, the network label mapping problem is to project the categorical information $\{\text{custom-character}.\text{sub}.*.\text{sup}.(1), \dots, \text{custom-character}.\text{sub}.*.\text{sup}.(N)\}$ to a code in a code alphabet custom-character . This is an encoding problem, i.e., to find an encoder:

$$[00001]f: \text{Math}._n \in \text{sup}^{(n)} \rightarrow \text{fwdarw} \cdot$$

Because the $\text{custom-character}.\text{sub}.*.\text{sup}.(n), \forall n$ contains categorical information, one may use a known categorical embedding technique.

Marker Aware Policy

[0163] At least a part of the network policies may comprise importance marker-awareness to prioritize the packets/flows from microservices with higher importance levels, as indicated by the respective importance marker. There may be different strategies to prioritize or weight the network performance based on the importance marker and introduce these strategies into the options of network policy, as shown in module B in FIG. 1. In this way, the information of the chosen importance marker-dependent strategy is also embedded in the network label and provided to the application for better optimization of the importance level assignment.

[0164] In an example, the RAN scheduling policy is extended with importance marker awareness. A similar extension can be easily done for the other policies. The RAN scheduling strategy has two conventional options: round-robin (RR) and proportional fairness (PF). RR handles all packets/flows without priority, also known as cyclic executive. To introduce into RR the priority of the microservices with higher importance level, they may be served with fixed priority rank based on the importance levels. E.g., importance levels 3, 2, 1 are defined with the priority rank 1, 2, 3, respectively. For conventional PF, the channel is scheduled based on the priority function $P=T.\text{sup}.\alpha/R.\text{sup}.\beta$ where T denotes the data rate potentially achievable in the present time slot, while R is the historical average data rate, and α and β are the tuning factors for adjusting the fairness. This formulation adjusts the balance between serving the best terminals with best channel conditions and serving the terminals with poor performance such that they can reach an acceptable level of performance. To introduce the importance marker awareness, we can define a modified priority $P'=g(P, I)$ which is monotonically increasing with the importance level/for any fixed priority P . Different functions can be defined, such as affine functions where $P'=aP+b$ and a, b are non-negative real numbers.

IMO Implementation

[0165] The IMO's definition and usage are described with respect to actions 3, 5, and 8. The IMO may be implemented as a plug-in add-on in the API gateway. The provider of the add-on may be the CSP, for example. As described, the IMO serves as a lightweight interface between the Application API and the network system. On one hand, it receives feedback from the network by means of the extracted network labels; on the other hand, it receives microservices' information and users' feedback from the API gateway. Since it is an add-on in the API gateway, the application is responsible for its own importance level assignment of the microservices, and network only passively reacts to it. This helps protect the privacy of the application. The network also benefits from the embedded importance markers of the packets/flows, as it allows a more efficient resource and energy utilization for satisfying the users' QoE.

[0166] As an example embodiment, the IMO for importance marker assignment may be designed with reinforcement learning agent as shown in Module C in FIG. 1. [0167] The state is composed of 1) network label indicating the network policies, and 2) microservices' QoS requirements. For

the second embodiment we can add 3) QoS requirements of the inter-microservice communications. [0168] The action is the assigned importance marker of each microservice. For inter-microservice communications, the action also includes the importance marker of the inter-microservice communication. [0169] The reward is defined as the goal-oriented users' QoE. [0170] Various reinforcement learning methods can be used, for example, the actor-critic algorithms. The actor-critic algorithm combines both policy-based methods (actor) and value-based methods (critic). In particular, the actor learns a policy to make decisions on the action, based on the observed state, and the critic evaluates the action taken by the actor for given state by estimating the value function. An example embodiment of an architecture of the actor-critic deep reinforcement learning model for the optimization of the importance level assignment is shown in FIG. 4.

[0171] Some example embodiments provide at least one of the following advantages: [0172] Goal-oriented and microservice-aware: The microservice-specific importance marker helps the network to adapt its policies towards the application's goal by providing dedicated or prioritized performance for the microservices that are the most critical for the user's QoE. [0173] Efficient resource and energy utilization: Network adapts its optimization policies based on the importance markers, preventing the resource over-provision, and encouraging the energy saving, e.g., less critical microservices can be allocated with less resource and requires less frequently retransmission without affecting the user's QoE. [0174] Dual-side privacy-preserving: 1) Application optimizes and assigns the importance markers to its microservices, while network does not directly intervene. 2) Network maps its policies to label, and application does not know the details of the network's configuration. [0175] Lightweight network-application interaction: The exchanged information includes only a higher layer extraction of network label and the embedded importance marker in the packets/flows. Moreover, the interaction is realized via IMO as a lightweight plug-in add-on to the API gateway.

[0176] Some example embodiments may be applied to a wide range of applications built by multiple microservices with different QoS requirements such as: [0177] Cloud Gaming: the microservices in charge of fetching the physical interaction of the gamer with the game controllers will deal with real-time, high frequency, small size data, while the microservices in charge of computing the visualization of the game necessitate access to dedicated hardware and the microservices streaming the visual state of the game require high bandwidth. [0178] AR/VR applications: VR application will share many similarities with cloud gaming applications. The different microservices dealing with the control, visualization and streaming will have very different needs. AR applications can be more versatile in terms of mode of operation and the level of added information. Nevertheless, their different sub-components are bound to be very unbalanced in term of upstream and downstream network needs. [0179] Vertical industrial applications: some microservices will gather lots of heterogenous data from physical sensors (leading to many combinations of frequency and size of reporting data), while automation microservices will apply control loops algorithms and issue control commands to physical actuators. Some of the controls need to be real time (for instance the safety related ones), while others can stand to have a longer time window (like small performance configuration adjustments). [0180] As an example, it is shown how the framework according to some example embodiments may be applied to a streaming application, such as Netflix.

[0181] The Netflix's service is decomposed into many microservices. Some of them are shown in Table 1, together with their description and QoS requirements.

TABLE-US-00001 TABLE 1 Example microservices of streaming applications and their QoS requirements.

Microservice	Description	QoS Requirements
Subscribe-service	For users' subscription	Throughput: low Latency: medium Security/Reliability: high
Auth-service	For authentication	Throughput: low process of the system, Latency: low auth-user retrieve, Security/Reliability: high auth-user store.
Product-service	For movie store and listing.	Throughput:

high Latency: high Security/Reliability: low Order/Billing- For movie ordering and Throughput: low service billing. Latency: low Security/Reliability: high Search-service For searching products Throughput: medium and categories. Latency: low Security/Reliability: medium Recommendation- For movie Throughput: medium service recommendations. Latency: medium Security/Reliability: low Streaming-service For video streaming. Throughput: high Latency: low Security/Reliability: low Caching-service For caching the most Throughput: medium popular or most likely to be Latency: high watched contents by users Security/Reliability: low Review-service For review process and Throughput: low censoring the reviews. Latency: high Security/Reliability: medium [0182] The users' QoE can be computed based on the following metrics: [0183] Rebuffer rate: a measure of how often playback is temporarily interrupted while more data is downloaded from the server to replenish the local buffer on the client device. [0184] Bitrate: the quality of the picture that is served/watched, e.g., a low bitrate results in a fuzzy picture. [0185] Other intrinsic metrics related to viewing experience. [0186] Users' direct feedback or rating. [0187] Other analytics including active subscribers, subscriber growth rate, churn rate, content hours streamed, etc. [0188] An IMO add-on, as shown in FIG. 1 and FIG. 2 can be added to the Netflix system, such that the importance markers are automatically learned and assigned to the microservices, based on the extracted network label provided by the network, the microservices' QoS requirements known to the API gateway, and the computed users' QoE based on the application's interest. The framework is shown in FIG. 5. Note that, for brevity, the details at the network system side are omitted, but the architecture, functions, and message flows are similar to those shown in FIG. 2, and the IMO's algorithm can have a similar design as described hereinabove with respect to FIG. 4. [0189] FIG. 6 shows an apparatus according to an example embodiment. The apparatus may be an optimizer (such as an IMO) or an element thereof. FIG. 7 shows a method according to an example embodiment. The apparatus according to FIG. 6 may perform the method of FIG. 7 but is not limited to this method. The method of FIG. 7 may be performed by the apparatus of FIG. 6 but is not limited to being performed by this apparatus.

[0190] The apparatus comprises first means for receiving **110**, second means for receiving **120**, third means for receiving **130**, means for determining **140**, means for marking **150**, and means for providing **160**. The first means for receiving **110**, second means for receiving **120**, third means for receiving **130**, means for determining **140**, means for marking **150**, and means for providing **160** may be a first receiving means, second receiving means, third receiving means, determining means, marking means and providing means, respectively. The first means for receiving **110**, second means for receiving **120**, third means for receiving **130**, means for determining **140**, means for marking **150**, and means for providing **160** may be a first receiver, second receiver, third receiver, determiner, marker, and provider, respectively. The first means for receiving **110**, second means for receiving **120**, third means for receiving **130**, means for determining **140**, means for marking **150**, and means for providing **160** may be a first receiving processor, second receiving processor, third receiving processor, determining processor, marking processor, and providing processor, respectively.

[0191] The first means for receiving **110** receives, for each of plural microservices, a respective requirement of a quality of service (**S110**). The plural microservices jointly build an application.

[0192] The second means for receiving **120** receives a label (**S120**). The label indicates a policy status of a communication system conveying data packets from the microservices for the application.

[0193] The third means for receiving **130** receives an indication of a quality of experience of the application (**S130**).

[0194] The means for determining **140** determines, for a first one of the microservices, an importance level (**S140**). The determination is based on the requirement of the quality of service of the first one of the microservices, the label, and the quality of experience.

[0195] The means for marking **150** marks a data packet by the importance level determined for the

first one of the microservices (**S150**). The data packet is originated from or designated to the first one of the microservices and to be conveyed by the communication system.

[0196] The means for providing **160** provides the data packet marked by the importance level to the communication system for conveying the data packet (**S160**).

[0197] FIG. **8** shows an apparatus according to an example embodiment. The apparatus may be communication system (such as a network) or an element thereof (e.g. a NSMF or a NSSMF, or an element of one of these network functions). FIG. **9** shows a method according to an example embodiment. The apparatus according to FIG. **8** may perform the method of FIG. **9** but is not limited to this method. The method of FIG. **9** may be performed by the apparatus of FIG. **8** but is not limited to being performed by this apparatus.

[0198] The apparatus comprises means for receiving **210**, means for modifying **220**, and means for informing **230**. The means for receiving **210**, means for modifying **220**, and means for informing **230** may be a receiving means, modifying means, and informing means, respectively. The means for receiving **210**, means for modifying **220** and means for informing **230** may be a receiver, modifier, and informer, respectively. The means for receiving **210**, means for modifying **220** and means for informing **230** may be a receiving processor, modifying processor, and informing processor, respectively.

[0199] The means for receiving **210** receives, from an optimizer, a data packet for conveying the data packet by a communication system (**S210**). The data packet is marked by an importance level.

[0200] The means for modifying **220** modifies a policy status of the communication system based on the importance level of **S210** (**S220**).

[0201] The means for informing **230** informs the optimizer by a label on the policy status of the communication system after the policy status of the communication system was modified by the means for modifying **220** (**S230**).

[0202] FIG. **10** shows an apparatus according to an example embodiment. The apparatus comprises at least one processor **810**, at least one memory **820** storing instructions that, when executed by the at least one processor **810**, cause the apparatus at least to perform the method according to at least one of the following figures and related description: FIG. **7** or FIG. **9**.

[0203] In some example embodiments, the packets from/to all the microservices are marked by the respective importance marker. However, in some example embodiments, only packets from/to a subset of the microservices are marked by the respective importance marker. For packets not marked by the respective importance marker, the network may apply a default policy. For example, the default policy may be a handling with low priority (e.g. large latency, low throughput, etc.). The provider of the application may decide that some of the microservices have a very low importance for the QoE such that the network should treat them always with low priority. For these microservices, IMO may not determine any importance marker and their packets may remain without importance marker.

[0204] Some example embodiments are explained with respect to 6G. However, other example embodiments may be employed in other 3GPP generations, such as 4G, 5G, 7G, etc., or in other environments where data blocks as a whole are transmitted in respective transport blocks.

[0205] One piece of information may be transmitted in one or plural messages from one entity to another entity. Each of these messages may comprise further (different) pieces of information.

[0206] Names of network elements, network functions, protocols, and methods are based on current standards, or are current proposals. These names are not limiting. For example, in other versions or other technologies, the names of these network elements and/or network functions and/or protocols and/or methods may be different, as long as they provide a corresponding functionality. The same applies correspondingly to the terminal.

[0207] If not otherwise stated or otherwise made clear from the context, the statement that two entities are different means that they perform different functions. It does not necessarily mean that they are based on different hardware. That is, each of the entities described in the present

description may be based on a different hardware, or some or all of the entities may be based on the same hardware. It does not necessarily mean that they are based on different software. That is, each of the entities described in the present description may be based on different software, or some or all of the entities may be based on the same software. Each of the entities described in the present description may be deployed in the cloud.

[0208] According to the above description, it should thus be apparent that example embodiments provide, for example, an optimizer (such as an Importance Marker Optimizer) or an element thereof (which may or may not be actually integrated in the optimizer), an apparatus embodying the same, a method for controlling and/or operating the same, and computer program(s) controlling and/or operating the same as well as mediums carrying such computer program(s) and forming computer program product(s). According to the above description, it should thus be apparent that example embodiments provide, for example, a communication system (such as a 3GPP system) or an element thereof (which may or may not be actually integrated in the communication system), an apparatus embodying the same, a method for controlling and/or operating the same, and computer program(s) controlling and/or operating the same as well as mediums carrying such computer program(s) and forming computer program product(s).

[0209] Implementations of any of the above-described blocks, apparatuses, systems, techniques or methods include, as non-limiting examples, implementations as hardware, software, firmware, special purpose circuits or logic, general purpose hardware or controller or other computing devices, or some combination thereof. Each of the entities described in the present description may be embodied in the cloud.

[0210] It is to be understood that what is described above is what is presently considered the preferred example embodiments. However, it should be noted that the description of the preferred example embodiments is given by way of example only and that various modifications may be made without departing from the scope of the disclosure as defined by the appended claims.

[0211] The terms “first X” and “second X” include the options that “first X” is the same as “second X” and that “first X” is different from “second X”, unless otherwise specified. As used herein, “at least one of the following: <a list of two or more elements>” and “at least one of <a list of two or more elements>” and similar wording, where the list of two or more elements are joined by “and” or “or”, mean at least any one of the elements, or at least any two or more of the elements, or at least all the elements. The term “or” refers to a non-exclusive “or” unless otherwise indicated (e.g., use of “or else” or “or in the alternative”).

Claims

1. Apparatus comprising: at least one processor; at least one memory storing instructions that, when executed by the at least one processor, cause the apparatus at least to perform: receiving, for each of plural microservices, a respective requirement of a quality of service, wherein the plural microservices jointly build an application; receiving a first label, wherein the first label indicates a policy status of a communication system conveying data packets from the microservices for the application; receiving an indication of a first quality of experience of the application; determining, for a first one of the microservices, an importance level based on the requirement of the quality of service of the first one of the microservices, the first label, and the first quality of experience, marking a data packet originated from or designated to the first one of the microservices and to be conveyed by the communication system by the importance level determined for the first one of the microservices; providing the data packet marked by the importance level to the communication system for conveying the data packet.

2. The apparatus according to claim 1, wherein the determining comprises determining, for each of the microservices, a respective importance level based on the requirement of the quality of service of the respective microservice, the first label, and the first quality of experience, and the marking

comprises marking, for each of the microservices, a data packet originated from or designated to the respective microservice and to be conveyed by the communication system by the importance level determined for the respective microservice.

3. The apparatus according to claim 1, wherein the determining comprises determining an inter-microservice importance level based on the requirement of the quality of service of the first one of the microservices, the requirement of the quality of service of a second one of the microservices, the first label, and the first quality of experience; and the marking comprises marking an inter-microservices data packet by the inter-microservice importance level, wherein the inter-microservices data packet is to be conveyed by the communication system between the first one of the microservices and the second one of the microservices.

4. The apparatus according to claim 1, wherein the importance level determined for the first one of the microservices is included in a header of the data packet originated from or designated to the first one of the microservices and to be conveyed by the communication system.

5. The apparatus according to claim 1, wherein the data packet originated from or designated to the first one of the microservices and to be conveyed by the communication system belongs to a traffic flow, and the traffic flow is marked by the importance level.

6. The apparatus according to claim 1, further comprising receiving, from the communication system in response to the providing the data packet marked by the importance level to the communication system, a second label indicating a policy status of the communication system after the communication system evaluated the importance level; receiving, in response to the providing the data packet marked by the importance level to the communication system, an indication of a second quality of experience of the application; adapting the determining by the means for determining based on the first label, the second label, the requirements of the quality of service of each of the microservices, the first quality of experience, and the second quality of experience for optimizing the quality of experience of the application.

7. The apparatus according to claim 6, wherein the adapting comprises adapting the determining by machine learning.

8. The apparatus according to claim 7, wherein the machine learning comprises at least one of reinforcement learning and supervised learning.

9. Method comprising receiving, for each of plural microservices, a respective requirement of a quality of service, wherein the plural microservices jointly build an application; receiving a first label, wherein the first label indicates a policy status of a communication system conveying data packets from the microservices for the application; receiving an indication of a first quality of experience of the application; determining, for a first one of the microservices, an importance level based on the requirement of the quality of service of the first one of the microservices, the first label, and the first quality of experience, marking a data packet originated from or designated to the first one of the microservices and to be conveyed by the communication system by the importance level determined for the first one of the microservices; providing the data packet marked by the importance level to the communication system for conveying the data packet.

10. The method according to claim 9, wherein, for each of the microservices, a respective importance level is determined based on the requirement of the quality of service of the respective microservice, the first label, and the first quality of experience, and, for each of the microservices, a data packet originated from or designated to the respective microservice and to be conveyed by the communication system is marked by the importance level determined for the respective microservice.

11. The method according to claim 9, wherein an inter-microservice importance level is determined based on the requirement of the quality of service of the first one of the microservices, the requirement of the quality of service of a second one of the microservices, the first label, and the first quality of experience; and an inter-microservices data packet is marked by the inter-microservice importance level, wherein the inter-microservices data packet is to be conveyed by

the communication system between the first one of the microservices and the second one of the microservices.

12. The method according to claim 9, wherein the importance level determined for the first one of the microservices is included in a header of the data packet originated from or designated to the first one of the microservices and to be conveyed by the communication system.

13. The method according to claim 9, wherein the data packet originated from or designated to the first one of the microservices and to be conveyed by the communication system belongs to a traffic flow, and the traffic flow is marked by the importance level.

14. The method according to claim 9, further comprising receiving, from the communication system in response to the providing the data packet marked by the importance level to the communication system, a second label indicating a policy status of the communication system after the communication system evaluated the importance level; receiving, in response to the providing the data packet marked by the importance level to the communication system, an indication of a second quality of experience of the application; adapting the determining the importance level based on the first label, the second label, the requirements of the quality of service of each of the microservices, the first quality of experience, and the second quality of experience for optimizing the quality of experience of the application.

15. The method according to claim 14, wherein the determining the importance level is adapted by machine learning.

16. A computer readable medium comprising a set of instructions which, when executed on an apparatus, is configured to cause the apparatus to carry out: receiving, for each of plural microservices, a respective requirement of a quality of service, wherein the plural microservices jointly build an application; receiving a first label, wherein the first label indicates a policy status of a communication system conveying data packets from the microservices for the application; receiving an indication of a first quality of experience of the application; determining, for a first one of the microservices, an importance level based on the requirement of the quality of service of the first one of the microservices, the first label, and the first quality of experience, marking a data packet originated from or designated to the first one of the microservices and to be conveyed by the communication system by the importance level determined for the first one of the microservices; providing the data packet marked by the importance level to the communication system for conveying the data packet.
